

Laboratorio de Filogeografía: Género *Hamadryas*

Replica del artículo Vega-Sanchez et al., 2019

Laboratorio de Filogeografía

2026-03-18

Tabla de contenidos

1	Introducción	1
1.1	Objetivos	2
2	Preparación de datos de entrada	2
2.1	Paquetes necesarios	2
2.1.a	Paquetes que vale la pena estudiar para análisis genéticos	4
2.2	Organización del proyecto	4
2.3	Flujo de trabajo general	6
2.4	Descarga de los datos	7
2.4.a	Descargar metadatos	10
2.4.b	Unir metadatos geográficos	10
2.5	Alineamiento	12
2.5.a	Inspección visual del alineamiento	13
2.6	Identificación de haplotipos	14
3	Análisis	15
3.1	Diversidad haplotípica y nucleotídica	15
3.2	Diferenciación genética	16
3.2.a	Análisis de varianza molecular AMOVA	17
3.2.b	Redes de haplotipos	19
3.2.c	Mapa	23
3.3	Tip de reproducibilidad	25

1 Introducción

Vamos a realizar una práctica de análisis básicos de filogeografía, tratando de replicar algunos de los resultados del artículo de Vega-Sánchez et al., 2019. En este artículo se utilizaron secuencias COI e ITS de libélulas de la especie *Hetaerina americana*.

i Preguntas

1. ¿Cuáles eran las preguntas de investigación del artículo?
2. ¿Cuál es el resultado principal?

Nos enfocaremos en algunos de a análisis que se realizaron con las secuencias del COI.

1.1 Objetivos

1. Descargar las secuencias
2. Obtener metadatos geográficos
3. Alinear secuencias y cortar regiones con datos faltantes
4. Identificar haplotipos
5. Obtener medidas básicas de diversidad genética
6. Explorar la estructura genética mediante un análisis AMOVA
7. Explorar las relaciones entre haplotipos mediante una red de haplotipos
8. Comparar las relaciones en la red con su distribución en el mapa.

2 Preparación de datos de entrada

2.1 Paquetes necesarios

```
# Instalar paquetes desde CRAN
install.packages(c(
  "bold",
  "rentrez",
  "ape",
  "seqinr",
  "stringr",
  "dplyr",
  "tidyr",
  "ggplot2",
  "knitr",
  "kableExtra",
  "here",
  "fs",
  "maps",
  "pegas",
  "adegenet",
  "poppr",
  "RColorBrewer",
  "maps",
  "scatterpie",
  "gtools"
```

```

))

# Instalar paquetes desde Bioconductor
if (!require("BiocManager", quietly = TRUE)) {
  install.packages("BiocManager")
}

BiocManager::install(c(
  "Biostrings",
  "msa"
))

```

```

library(bold)      # Herramientas para descargar secuencias de la base de datos
BOLD (Barcode of Life)
library(rentrez)    # Interfaz para consultar y descargar secuencias de NCBI
(GenBank)

library(ape)        # Manipulación de secuencias y análisis filogenéticos (DNABin,
distancias genéticas, árboles)
library(Biostrings) # Manejo eficiente de secuencias biológicas (DNABin,
lectura y manipulación de FASTA)
library(msa)        # Alineamiento múltiple de secuencias (MUSCLE, ClustalW,
ClustalOmega)
library(pegas)      # Análisis de genética poblacional y filogeografía
(haplotipos, redes haplotípicas)
library(ade4)       # Análisis multivariado de datos genéticos (estructuración
poblacional, objetos genéticos)
library(poppr)      # Análisis de diversidad genética y clonación en poblaciones

library(stringr)    # Manipulación de cadenas de texto (nombres de secuencias,
metadatos)
library(dplyr)      # Manipulación y filtrado de tablas de datos
library(tidyverse)  # Reorganización de datos (pivotar, separar o combinar
columnas)
library(gtools)     # Funciones auxiliares para manipulación de datos (por
ejemplo ordenamiento natural)

library(ggplot2)    # Visualización de datos y generación de gráficos
library(maps)       # Herramientas de mapas
library(RColorBrewer) # Paletas de colores para visualización de datos
library(scatterpie) # Gráficos de pastel sobre mapas o coordenadas (ej.
frecuencias de haplotipos)

```

```
library(knitr)      # Generación de tablas y reportes reproducibles
library(kableExtra) # Mejora estética de tablas en HTML o PDF

library(here)      # Manejo de rutas relativas dentro de proyectos reproducibles
library(fs)        # Manejo de archivos y directorios (crear, listar, verificar
rutas)
```

2.1.a Paquetes que vale la pena estudiar para análisis genéticos

- Adegnet
- Biostrings
- pegas
 - Hay un libro de texto que se llama Population genetics with R con el que pueden practicar
- ape
 - Hay un libro de texto que se llama Analysis of Phylogenetics and Evolution with R

¿Qué estudiar?

- ¿Cuál es la estructura de los objetos principales de cada paquete?
- ¿Cómo puedo manipular las distintas partes?
- ¿Cómo transformar entre ellos?
- ¿Qué cosas puedo hacer con cada uno?

2.2 Organización del proyecto

Una de las decisiones más importantes al iniciar un proyecto es cómo organizarlo. Una buena estructura permite que los análisis sean claros, estén bien documentados y, sobre todo, sean reproducibles. Aunque existen muchas formas de organizar un proyecto, y en gran medida depende de las preferencias personales y del equipo de trabajo, hay algunos principios generales que conviene seguir.

Primero, es fundamental conservar los datos crudos en un directorio separado. Esto permite volver al punto de partida en cualquier momento y asegura que todo el flujo de trabajo pueda reproducirse desde cero.

Segundo, se puede estructurar las carpetas para

Segundo, la estructura del proyecto debe estar claramente documentada. Cualquier persona (incluyéndote en el futuro) debería poder entender rápidamente dónde están los datos, los scripts y los resultados.

Tercero, todos los pasos del análisis, así como las decisiones tomadas durante el proceso, deben registrarse. Esto no solo facilita la reproducibilidad, sino también la revisión, corrección y extensión del trabajo.

Tip de reproducibilidad

Un aspecto clave de la reproducibilidad es asegurarse de que otras personas (o tú misma en el futuro) puedan ejecutar el código en el mismo entorno en el que fue desarrollado. En R, esto implica controlar las versiones de los paquetes utilizados.

Para ello, una herramienta muy útil es el paquete `renv`, que permite crear entornos aislados y registrar las versiones exactas de las dependencias del proyecto.

```
# install.packages("renv")

library(renv)
renv::init()
```

Estructura básica de directorios

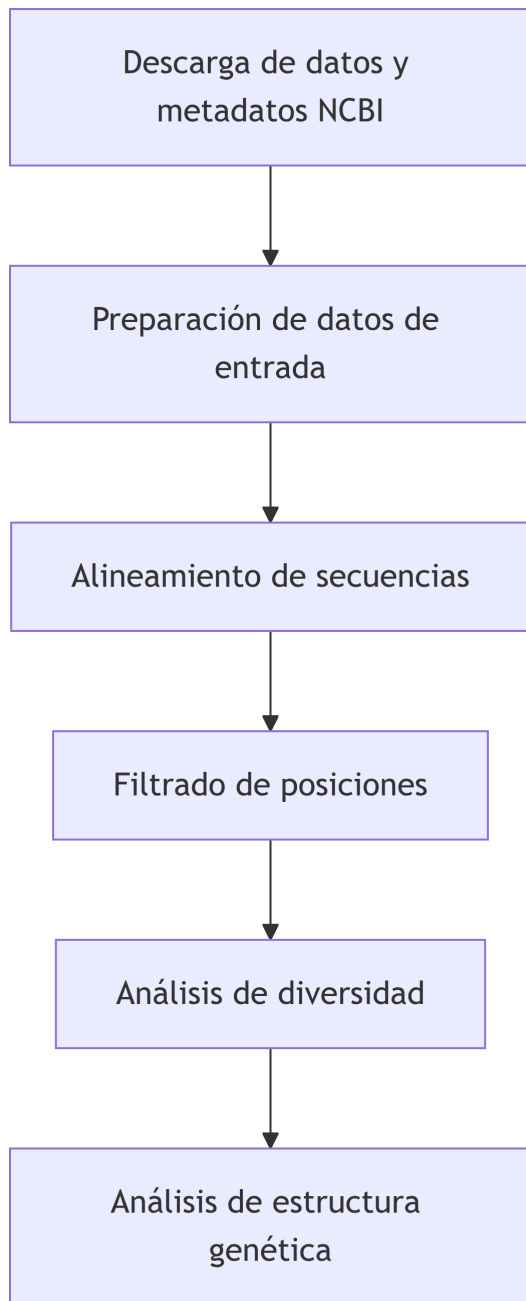
A continuación, generaremos una estructura de directorios sencilla como punto de partida:

```
hetaerina_lab
├─ 00_raw_data
├─ 01_metadata
├─ 02_alignment
└─ 03_outputs
```

```
# Crear estructura de carpetas del proyecto
dirs <- c(
  "hetaerina_lab/00_raw_data",
  "hetaerina_lab/01_metadata",
  "hetaerina_lab/02_alignment",
  "hetaerina_lab/03_outputs"
)

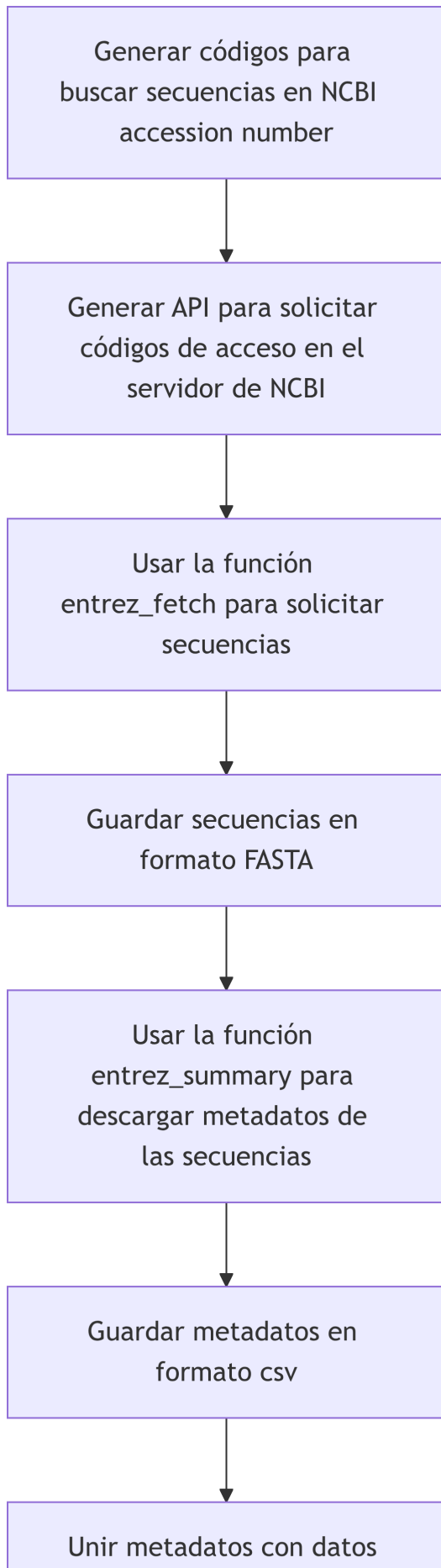
for (d in dirs) {
  if (!dir.exists(d)) dir.create(d, recursive = TRUE)
}
```

2.3 Flujo de trabajo general



Flujo de trabajo para la descarga de datos

2.4 Descarga de los datos



Flujo de trabajo para la descarga de datos

En el material suplementario del artículo, en la tabla S1 podemos encontrar los números de acceso de las secuencias COI e ITS de las libélulas

Primero vamos a generar un vector con todos los números de acceso de *H. americana*

Nota: Solo vamos a trabajar con las localidades con números de acceso de GeneBank. Las otras secuencias las obtuvieron de otra base de datos llamada BOLD.

```
# H. americana - COI
# MH084732-MH084741, and MH090478-MH090669

acc1 <- paste0("MH", sprintf("%06d", 84732:84741))

acc2 <- paste0("MH", sprintf("%06d", 90478:90669))

acc_COI <- c(acc1, acc2)
acc_ITS <- paste0("MH", sprintf("%06d", 90377:90477))
```

Para descargar secuencias de GeneBank se utiliza el paquete `rentrez`. Necesitan crear una cuenta en GeneBank para poder generar una clave API que les permita descargar las secuencias.

```
ncbi_key <- Sys.getenv("NCBI_API_KEY") # Ustedes pueden sustituir este código
por ncbi_key <- "API_KEY"

rentrez::set_entrez_key(ncbi_key)

fasta_lote_COI <- rentrez::entrez_fetch(
  db = "nucleotide",
  id = acc_COI,
  rettype = "fasta"
)

write(
  fasta_lote_COI,
  file = file.path(
    here(),
    "hetaerina_lab/00_raw_data/raw_H_americana_COI.fasta"
  )
)

# Código por si quieren descargar las secuencias ITS y probar
# fasta_lote_ITS <- rentrez::entrez_fetch(
#   db = "nucleotide",
```

```

#   id = acc_ITS,
#   rettype = "fasta"
# )

# write(
#   fasta_lote_ITS,
#   file = file.path(
#     here(),
#     "hetaerina_lab/00_raw_data/raw_H_americana_ITS.fasta"
#   )
# )

```

2.4.a Descargar metadatos

```

metadatos_COI_raw <- rentrez::entrez_summary(db = "nucleotide", id = acc_COI)

metadata_COI_ls <- list()

for (rec in metadatos_COI_raw) {
  metadata_COI_ls[[rec$uid]] <- data.frame(
    accession = rec$accessionversion, # Numero de acceso
    title = rec$title, # encabezado de la secuencia
    taxid = rec$taxid, # id de la especie
    slen = rec$slen, # longitud de la secuencia
    organism = rec$organism # nombre del organismo
  )
}

metadata_COI <- bind_rows(metadata_COI_ls)

metadata_COI <- mutate(
  metadata_COI,
  locCode = str_extract(title, "(?<=Ham)[:alpha:]{1,}"),
  sampleID = as.numeric(str_extract(title, "(?<=Ham[[:alpha:]]{1,3})\\d{1,}")),
  sampleID = sprintf("%02d", sampleID)
)

```

2.4.b Unir metadatos geográficos

```

## Datos geograficos

geo_data <- read.table(
  "hetaerina_lab/01_metadata/metadata_geo.tsv",

```

```

    sep = "\t",
    header = T
)

metadata_COI <- read.csv(
  file = "hetaerina_lab/01_metadata/metadata_COI.csv",
  header = T
)

metadata_COI <- metadata_COI |> left_join(geo_data, by = c("locCode" = "Code"))

# Comparar con los tamaños de muestra originales
metadata_COI |> group_by(locCode) |> summarise(n = n(), n_orig = unique(N)) |
> View()

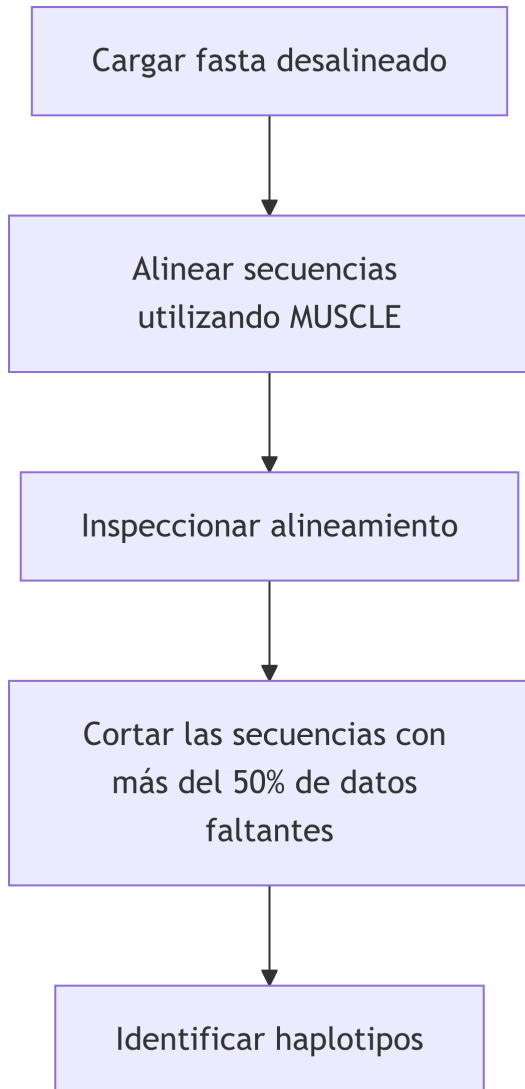
write.csv(
  metadata_COI,
  "hetaerina_lab/01_metadata/metadata_COI.csv",
  row.names = F
)

metadata_ITS <- metadata_ITS |> left_join(geo_data, by = c("locCode" = "Code"))

write.csv(
  metadata_ITS,
  "hetaerina_lab/01_metadata/metadata_ITS.csv",
  row.names = F
)

```

2.5 Alineamiento



Flujo de trabajo para alineamiento

```
coi      <-      Biostrings::readDNAStringSet("hetaerina_lab/00_raw_data/
raw_H_americana_COI.fasta")

# Ejecutar MUSCLE a través del paquete msa
coiAlign <- msa::msa(
  inputSeqs = coi,
  method    = "Muscle", # Alternativas: "ClustalW", "ClustalOmega"
  verbose   = FALSE,
```

```

    type = "dna"
  )

```

2.5.a Inspección visual del alineamiento

```

coiAlign_dnaBin <- msa::msaConvert(coiAlign, type = "ape::DNABin")
rownames(coiAlign_dnaBin) <- str_extract(
  rownames(coiAlign_dnaBin),
  "Ham\\w{1,}\\d{1,}"
)

```

```

pdf("hetaerina_lab/03_outputs/alignment_check.pdf", width = 10, height = 7, )
ape::checkAlignment(coiAlign_dnaBin, what = 1)
ape::checkAlignment(coiAlign_dnaBin, what = 2)
ape::checkAlignment(coiAlign_dnaBin, what = 3)
ape::checkAlignment(coiAlign_dnaBin, what = 4)
dev.off()

```

Recortar partes de las secuencias que tienen muchos datos faltantes

```

# Porcentaje de gaps por posición

aln_matrix <- as.character(coiAlign_dnaBin)

gaps_por_col <- apply(aln_matrix, 2, function(col) {
  mean(col == "-" | col == "n") * 100
})

cols_conservar <- which(gaps_por_col < 50)

coiAlign_trim <- coiAlign_dnaBin[, cols_conservar]

ape::write.dna(
  coiAlign_trim,
  file = "hetaerina_lab/02_alignment/alignment_coi_trimmed.fasta",
  format = "fasta"
)

ape::checkAlignment(coiAlign_trim, what = 1)

```

Number of sequences: 202

Number of sites: 629

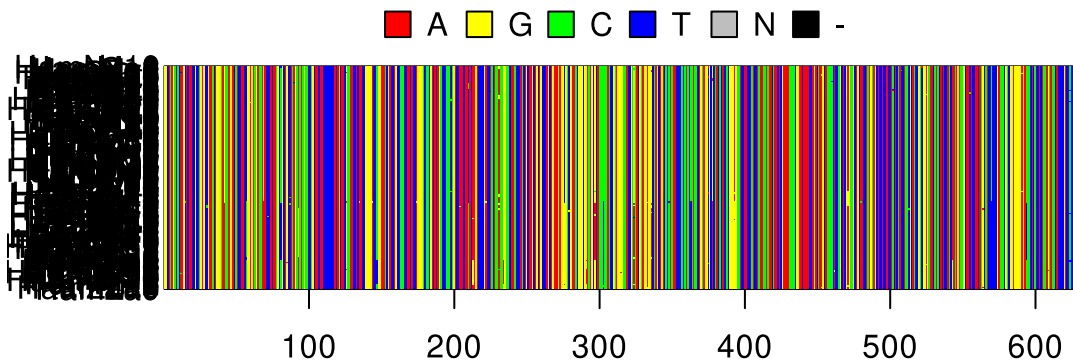
No gap in alignment.

Number of segregating sites (including gaps): 58

Number of sites with at least one substitution: 58

Number of sites with 1, 2, 3 or 4 observed bases:

1	2	3	4
571	51	6	1



2.6 Identificación de haplotipos

Recuperamos los haplotipos del alineamiento

```
coiAlign_trim <- ape::read.dna("hetaerina_lab/02_alignment/alignment_coi_trimmed.fasta", format = "fasta")
```

```
coiHaplo <- pegas::haplotype(coiAlign_trim)
coiHaplo
```

Haplotypes extracted from: coiAlign_trim

Number of haplotypes: 62
Sequence length: 629

Haplotype labels and frequencies:

I	II	III	IV	V	VI	VII	VIII	IX	X
1	1	1	1	1	1	1	1	2	3
XI	XII	XIII	XIV	XV	XVI	XVII	XVIII	XIX	XX
1	1	5	1	1	1	2	1	3	2
XXI	XXII	XXIII	XXIV	XXV	XXVI	XXVII	XXVIII	XXIX	XXX
1	1	80	1	1	3	1	3	1	1
XXXI	XXXII	XXXIII	XXXIV	XXXV	XXXVI	XXXVII	XXXVIII	XXXIX	XL
1	1	1	1	2	2	1	1	1	1
...									

(use summary() to print all)

3 Análisis

3.1 Diversidad haplotípica y nucleotídica

Para caracterizar la variación genética del conjunto de secuencias analizamos tres estadísticas comúnmente utilizadas en estudios de genética de poblaciones y filogeografía: la diversidad haplotípica (H_d), la diversidad nucleotídica (π) y la D de Tajima.

La diversidad haplotípica (H_d) mide la probabilidad de que dos secuencias elegidas al azar en la muestra correspondan a haplotipos distintos. Este estadístico refleja tanto el número de haplotipos presentes como la distribución de sus frecuencias en la muestra. Valores altos de H_d indican la presencia de múltiples haplotipos o una distribución relativamente equitativa de sus frecuencias.

La diversidad nucleotídica (π) representa el número promedio de diferencias nucleotídicas por sitio entre pares de secuencias de la muestra. Esta métrica proporciona una medida directa del grado de divergencia genética a nivel de nucleótidos dentro del conjunto de datos.

Finalmente, calculamos la D de Tajima, una prueba estadística que compara diferentes estimadores de la diversidad genética basados en el número de sitios segregantes y en las diferencias promedio entre secuencias. Este estadístico permite evaluar desviaciones del modelo neutral de evolución que pueden deberse a procesos demográficos (como expansiones poblacionales o cuellos de botella) o a la acción de la selección natural.

```
Hd <- pegas::hap.div(coiHaplo, variance=TRUE)
Pi <- pegas::nuc.div(coiHaplo, variance = TRUE)
Dtajima <- pegas::tajima.test(coiHaplo)
```

Hd

```
[1] 0.8134082065 0.0006113713
```

```
Pi
```

```
[1] 1.458456e-02 5.554843e-05
```

```
Dtajima
```

```
$D
```

```
[1] -0.1760188
```

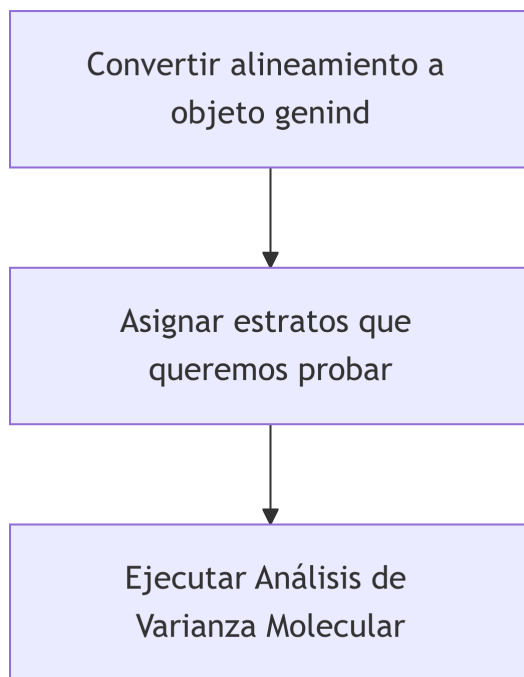
```
$Pval.normal
```

```
[1] 0.8602792
```

```
$Pval.beta
```

```
[1] 0.9013826
```

3.2 Diferenciación genética



Flujo de trabajo para estimar estructura genética (AMOVA)

```
coi_genind <- adegenet::DNABin2genind(coiAlign_trim)
```



```

metadata_COI <- read.csv("hetaerina_lab/01_metadata/metadata_COI.csv")
metadata_COI$locCode <- as.factor(metadata_COI$locCode)

adegenet::strata(coi_genind) <- metadata_COI
adegenet::setPop(coi_genind) <- ~locCode

coi_genind

```

```

/// GENIND OBJECT ///////////

// 202 individuals; 30 loci; 66 alleles; size: 168.1 Kb

// Basic content
@tab: 202 x 66 matrix of allele counts
@loc.n.all: number of alleles per locus (range: 2-4)
@loc.fac: locus factor for the 66 columns of @tab
@all.names: list of allele names for each locus
@ploidy: ploidy of each individual (range: 1-1)
@type: codom
@call: adegenet::DNAbin2genind(x = coiAlign_trim)

// Optional content
@pop: population of each individual (group size range: 1-10)
@strata: a data frame with 13 columns ( accession, title, taxid, slen,
organism, locCode, ... )

```

3.2.a Análisis de varianza molecular AMOVA

```

amova_pop <- poppr::poppr.amova(coi_genind, hier = ~locCode, within=FALSE,
method = "ade4")

amova_pop

```

```

$call
ade4::amova(samples = xtab, distances = xdist, structures = xstruct)

$results

```

	Df	Sum Sq	Mean Sq
Between samples	30	1597.3344	53.2444793
Within samples	171	157.3389	0.9201105
Total	201	1754.6733	8.7297177

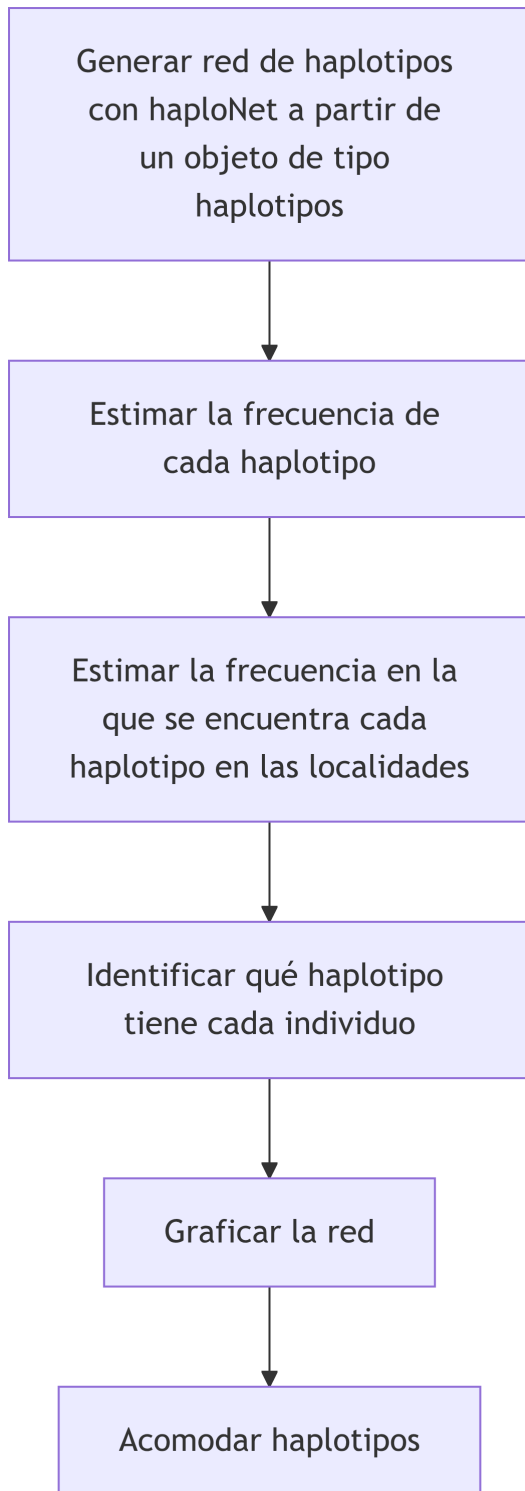
\$componentsofcovariance

		Sigma	%
Variations	Between samples	8.1258181	89.82846
Variations	Within samples	0.9201105	10.17154
Total variations		9.0459286	100.00000

\$statphi

	Phi
Phi-samples-total	0.8982846

3.2.b Redes de haplotipos



Flujo de trabajo para estimar estructura genética (red de haplotipos)

```

rownames(coiHaplo) <- as.numeric(as.roman(rownames(coiHaplo)))
red <- pegas::haploNet(coiHaplo)

freq_hap <- summary(coiHaplo)
names(freq_hap) <- as.numeric(as.roman(names(freq_hap)))

# ¿Qué haplotipo tiene cada individuo?
tmp <- stack(setNames(attr(coiHaplo, "index"), rownames(coiHaplo))) # recuperar
qué secuencia pertenece a qué haplotipo

ind_hap <- tmp %>%
  mutate(ind = as.integer(ind)) %>%
  group_by(values) %>%
  summarise(ind = list(ind), .groups = "drop") %>%
  tibble::deframe() |> unlist() # Agrupar los haplotipos usando como índice los
individuos, es decir, generar un objeto en el que podamos saber qué haplotipo
tiene cada individuo.

# Asignar colores a cada país
countries <- metadata_COI$Country # Vamos a colorear los haplotipos por país

countries <- factor(countries, levels = unique(countries))

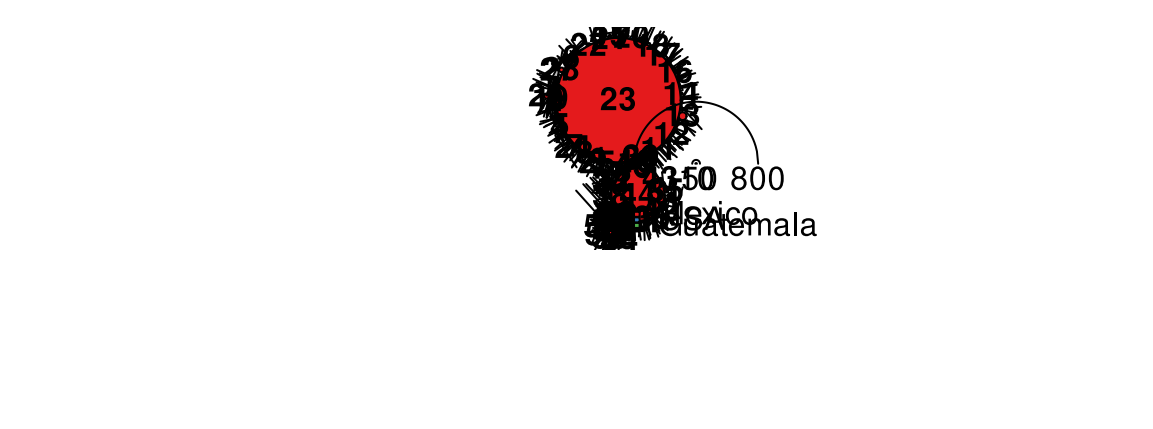
n_countries <- length(unique(countries)) # Número de localidades
paleta <- RColorBrewer::brewer.pal(min(n_countries, 8), "Set1") # Asignar un
color a cada país

# Matriz de proporción por haplotipo
prop_matriz <- ind_hap |>
  lapply(function(idx) table(countries[idx]) / length(idx)) |>
  bind_rows() |>
  as.matrix()

# Graficar
plot(
  red,
  size = freq_hap * 10, # tamaño de los haplotipos proporcional a su frecuencia
  pie = prop_matriz, # Asignar los colores por país (usando graficas de pie)
  bg = paleta,
  legend = c(100, -10),
  show.mutation = 1, # mostrar número de mutaciones en las ramas

```

```
main = "Red de haplotipos COI – Libélula"
```



Para modificar las coordenadas, en una terminal vamos a correr el script `mini_code_redhap.R`.
después vamos a modificar las coordenadas y guardarlas en un nuevo objeto.

```
# Primero abrir la terminal, abrir R y correr:  
source("mini_code_redhap.R") # carga la red
```

```
# Graficar
```

```
coords_manual <- readRDS(
  "hetaerina_lab/03_outputs/coords_red_haplotipos_modificadas.rds"
)
```

```
grupo3 <- c(31:46)
grupo4 <- c(47:62)
grupo2 <- c(29, 30)
grupo1 <- setdiff(1:62, c(grupo4, grupo2, grupo3))
```

```
# Función personalizada para generar paleta por grupo
asignar_colores <- function(haps, paleta_base) {
  colores <- grDevices::colorRampPalette(paleta_base)(length(haps))
  return(data.frame(hap = haps, color = colores))
}
```

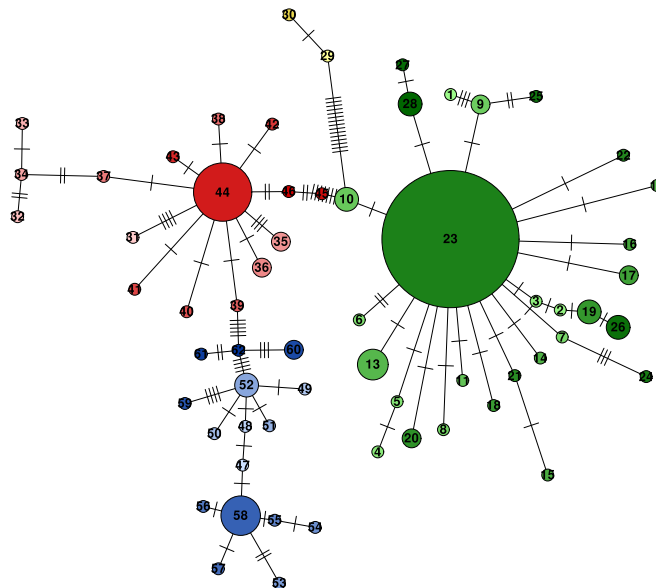
```

# Definir paletas base por grupo
colores_df <- bind_rows(
  asignar_colores(grupo1, c("#9cf989", "#006600")), # verdes
  asignar_colores(grupo2, c("#ffff99", "#e8d64e")), # amarillos
  asignar_colores(grupo3, c("#ffcccc", "#cc0000")), # rojos
  asignar_colores(grupo4, c("#cce0ff", "#003399")) # azules
)

mod_freq_hap <- ifelse(freq_hap > 30, freq_hap * 1.5, (log1p(freq_hap)) * 15)

plot(
  red,
  size = mod_freq_hap,
  bg = colores_df$color,
  xy = coords_manual,
  show.mutation = 1,
  main = "Red de haplotipos COI - Libélula",
  threshold = 0
)

```



```
pdf("hetaerina_lab/03_outputs/red_haplotipos_COI.pdf", width = 15, height = 10)
# Graficar
plot(
  red,
  size = mod_freq_hap, # tamaño de los haplotipos proporcional a su frecuencia
  bg = colores_df$color,
  xy = coords_manual,
  show.mutation = 1, # mostrar número de mutaciones en las ramas
  main = "Red de haplotipos COI – Libélula",
  threshold = 0
)
dev.off()
```

3.2.c Mapa

```
ind_hap_df <- data.frame(ind = 1:202, hap = ind_hap)
ind_hap_df <- ind_hap_df %>%
  mutate(
    gp = case_when(
      hap %in% grupo1 ~ "gp1",
      hap %in% grupo2 ~ "gp2",
      hap %in% grupo3 ~ "gp3",
      hap %in% grupo4 ~ "gp4"
    )
  ) %>%
  arrange(ind)

metadata_COI <- metadata_COI |> bind_cols(ind_hap_df)

# Calcular proporciones de haplotipos por localidad
pie_data <- metadata_COI %>%
  mutate(hap = paste0("hap", hap)) |>
  group_by(locCode, Lat, Lon, gp, hap) %>%
  summarise(n = n(), .groups = "drop") %>%
  group_by(locCode) %>%
  mutate(n = n / sum(n)) %>% # normalizar a proporciones
  ungroup() %>%
  pivot_wider(
    names_from = hap,
    values_from = n,
    values_fill = 0
  )
```

```

hap_cols_ordenados <- gtools::mixedsort(names(pie_data)[4:ncol(pie_data)])

pie_data <- as.data.frame(pie_data)[, c(
  names(pie_data)[1:3],
  hap_cols_ordenados
)]

# Vector de colores con nombres (hap → color)
colores_hap <- setNames(colores_df$color, paste0("hap", colores_df$hap))

# Columnas de haplotipos (todas menos locCode, lat, lon)
hap_cols <- colnames(pie_data)[startsWith(colnames(pie_data), "hap")]

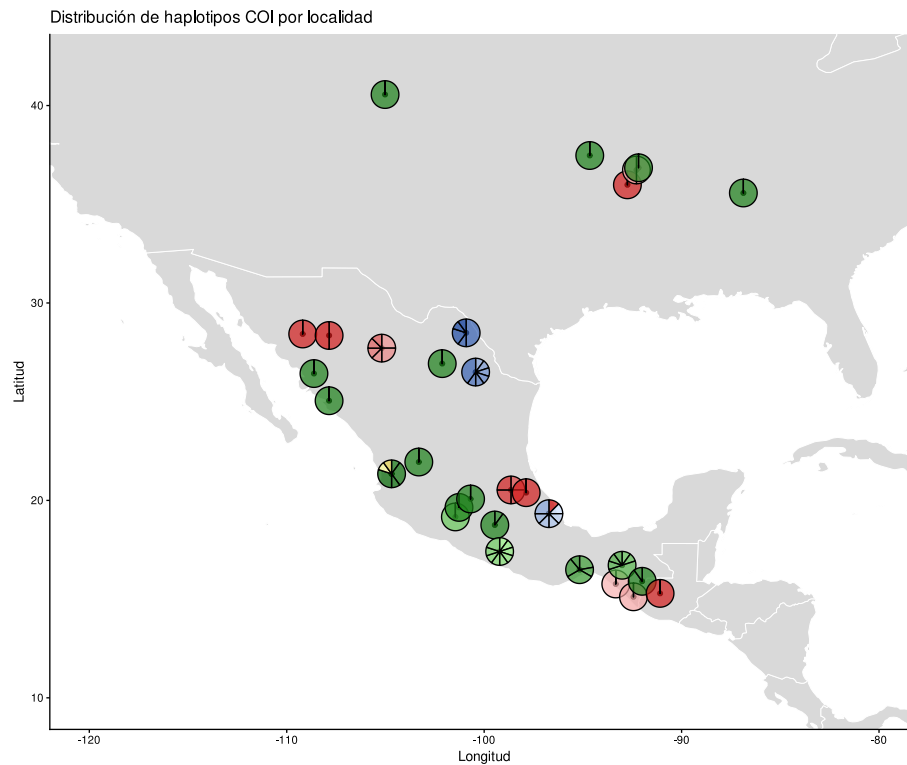
# Generar poligono del mundo
mapa_mundo <- ggplot2::map_data("world")

# Mapa
ggplot() +
  geom_polygon(
    data = mapa_mundo,
    aes(x = long, y = lat, group = group),
    fill = "grey85",
    color = "white",
    linewidth = 0.2
  ) +
  geom_point(data = pie_data, aes(x = Lon, y = Lat)) +
  scatterpie::geom_scatterpie(
    data = pie_data,
    aes(x = Lon, y = Lat, r = 0.7),
    alpha = 0.7,
    cols = colnames(pie_data)[startsWith(colnames(pie_data), "hap")],
    long_format = F
  ) +
  scale_fill_manual(values = colores_hap) +
  coord_fixed(xlim = c(-120, -80), ylim = c(10, 42)) +
  labs(
    title = "Distribución de haplotipos COI por localidad",
    x = "Longitud",
    y = "Latitud",
    fill = "Haplotipo"
  ) +

```



```
theme_classic(base_size = 13) +
theme(legend.position = "none")
```



3.3 Tip de reproducibilidad

Para guardar los paquetes que utilizamos

```
renv::snapshot()
```

```
lock <- renv::lockfile_read()

tabla <- data.frame(
  Package = names(lock$Packages),
  Version = sapply(lock$Packages, function(x) x$Version),
  row.names = NULL
)

knitr::kable(tabla)
```

Package	Version
BiocGenerics	0.54.1

Package	Version
BiocManager	1.30.27
BiocVersion	3.21.1
Biostrings	2.76.0
GenomeInfoDb	1.44.3
GenomeInfoDbData	1.2.14
IRanges	2.42.0
MASS	7.3-65
Matrix	1.7-4
R6	2.6.1
RColorBrewer	1.1-3
Rcpp	1.1.1
RcppArmadillo	15.2.3-1
S4Vectors	0.46.0
S7	0.2.1
UCSC.utils	1.4.0
XML	3.99-0.22
XVector	0.48.0
ade4	1.7-23
adeigenet	2.1.11
ape	5.8-1
askpass	1.2.1
base64enc	0.1-6
bold	1.3.0
boot	1.3-32
bslib	0.10.0
cachem	1.1.0
cli	3.6.5
cluster	2.1.8.2
commonmark	2.0.0
cpp11	0.5.3

Package	Version
crayon	1.5.3
crul	1.6.0
curl	7.0.0
data.table	1.18.2.1
digest	0.6.39
dplyr	1.2.0
evaluate	1.0.5
farver	2.1.2
fastmap	1.2.0
fontawesome	0.5.3
fs	1.6.7
generics	0.1.4
ggforce	0.5.0
ggfun	0.2.0
ggplot2	4.0.2
glue	1.8.0
gtable	0.3.6
gtools	3.9.5
here	1.0.2
highr	0.12
htmltools	0.5.9
httpcode	0.3.0
httpuv	1.6.16
httr	1.4.8
igraph	2.2.2
isoband	0.3.0
jquerylib	0.1.4
jsonlite	2.0.0
kableExtra	1.4.0
knitr	1.51

Package	Version
labeling	0.4.3
later	1.4.8
lattice	0.22-9
lifecycle	1.0.5
magrittr	2.0.4
maps	3.4.3
memoise	2.0.1
mgcv	1.9-4
mime	0.13
msa	1.40.0
nlme	3.1-168
openssl	2.3.5
otel	0.2.0
pegas	1.4
permute	0.9-10
pillar	1.11.1
pixmap	0.4-14
pkgconfig	2.0.3
plyr	1.8.9
polyclip	1.10-7
polysat	1.7-7
poppr	2.9.8
progressr	0.18.0
promises	1.5.0
purrr	1.2.1
rappdirs	0.3.4
rentrez	1.2.4
renv	1.1.8
reshape2	1.4.5
rlang	1.1.7

Package	Version
rmarkdown	2.30
rprojroot	2.1.1
rstudioapi	0.18.0
sass	0.4.10
scales	1.4.0
scatterpie	0.2.6
segmented	2.2-1
seqinr	4.2-36
shiny	1.13.0
sourcetools	0.1.7-1
sp	2.2-1
stringi	1.8.7
stringr	1.6.0
svglite	2.2.2
sys	3.4.3
systemfonts	1.3.2
textshaping	1.0.5
tibble	3.3.1
tidyr	1.3.2
tidyselect	1.2.1
tinytex	0.58
triebeard	0.4.1
tweenr	2.0.3
urltools	1.7.3.1
utf8	1.2.6
vctrs	0.7.1
vegan	2.7-3
viridisLite	0.4.3
withr	3.0.2
xfun	0.56

Package	Version
xml2	1.5.2
xtable	1.8-8
yaml	2.3.12
yulab.utils	0.2.4